

Multi-Agent Orchestrator

An engineering research report on Planner/executor DAG, tools,
self-correction

Sai Veda

Independent Project Research

2025

Contents

1. Abstract	3
2. Introduction	3
2.1 Research Problem	3
2.2 Research Questions and Hypotheses	3
3. Technical Background Review	4
3.1 Design Foundations	4
3.2 Supporting Examples	4
4. Research Method	4
5. Data Description	5
6. Analysis	5
6.1 Benchmark and Validation Results	5
6.2 Test Result Screenshots	5
6.3 Validation Checklist	6
7. Discussion	6
8. Conclusion	7
9. Future Work	7
10. References	7

1. Abstract

This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop. The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: Planner/executor DAG, tools, self-correction, implementation evidence, benchmark results, project validation

2. Introduction

Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly. A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries.

2.1 Research Problem

Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: Self-correction lifts task success 41.9% -> 100%.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 69 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 Task recovery uses execution feedback only, which keeps the success lift defensible. The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.
- 2 Planning and execution are intentionally decoupled so new tools do not rewrite orchestration logic.
- 3 The toolbox and accounting model keep the framework compatible with more capable planners later.

3.2 Supporting Examples

- Planner, scheduler, tool registry, guardrails, and accounting.
- Deterministic policy engine behind an LLM-like interface.
- Repair loop based on execution feedback and validator errors.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Defined the orchestrator around a DAG so ordering and parallelism stay explicit and testable. Added validators for task structure, tool arguments, and outputs before retries were allowed. Fed runtime failures back into planning so the system could revise rather than restart blindly. Benchmarked a 31-task suite to quantify the exact lift from self-correction.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 69 automated tests, 3 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

6.1 Benchmark and Validation Results

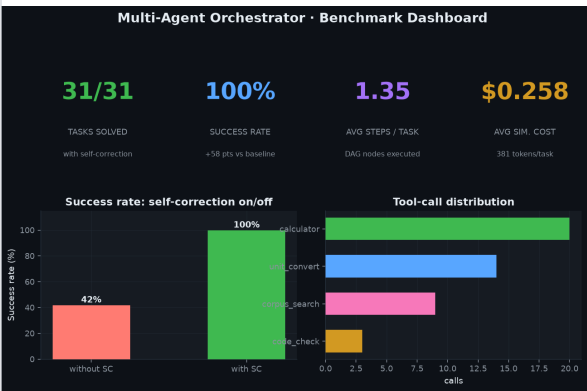
The headline repository result is reported as: Self-correction lifts task success 41.9% -> 100%. The table below reproduces benchmark evidence included in the repository where available.

Condition	Success rate	Solved	Avg steps	Avg attempts	Avg sim. cost
Without self-correction	41.9%	13/31	1.35	1.00	\$0.16524
With self-correction	100.0%	31/31	1.35	1.58	\$0.25834

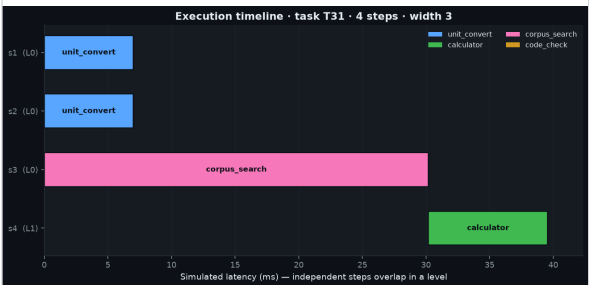
The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error. The measured cost-per-task makes the orchestration trade-off legible instead of qualitative.

6.2 Test Result Screenshots

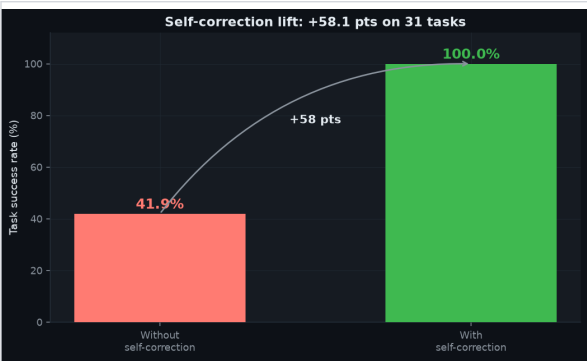
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



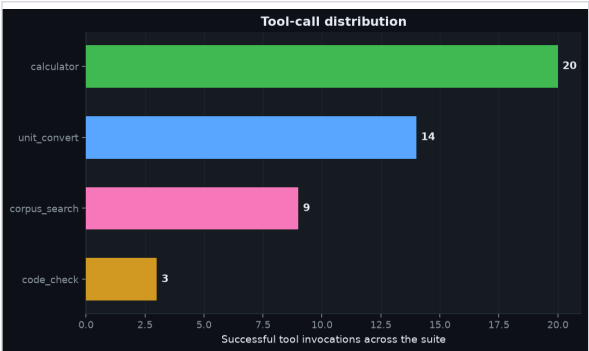
Kpi Dashboard



Execution Gantt



Success Rate



Tool Distribution

6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	69 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal. The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability. Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error. The measured cost-per-task makes the

orchestration trade-off legible instead of qualitative. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Multi-Agent Orchestrator is best understood as a complete technical implementation supported by repository-native evidence. This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop.

9. Future Work

- Replace the deterministic policy with a stronger planner while preserving the orchestration surface.
- Add long-horizon memory and tool confidence scoring for larger workflows.
- Instrument trace-level telemetry so repair behavior can be audited like any other workflow system.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/results.md - benchmark or result artefact used in the analysis section.
5. benchmarks/runner.py - benchmark or result artefact used in the analysis section.
6. benchmarks/tasks.py - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/execution_gantt.png - generated screenshot evidence included in the report.